

MPI 00.1: Parallel Trapezoidal Integration on Local and Expanse Clusters

Lakota Dolce

Department of Computing Sciences, Coastal Carolina University

Abstract—We implement and study a parallel trapezoidal rule integration using MPI, executing controlled parameter sweeps both on a local workstation and the SDSC Expanse system. We quantify performance via execution time, speedup, and efficiency while meeting course requirements: capturing local hardware characteristics, proving execution on Expanse, and analyzing strong-scaling behavior across process counts (P) and problem sizes (N). Our results show near-linear speedup for moderate P with diminishing efficiency at higher P due to communication overheads.

I. INTRODUCTION

This work follows the assignment brief for CSCI 473 which requires (i) local runs with hardware reporting, (ii) Expanse runs using the debug partition and the exact sweep command. We implement an MPI-based trapezoidal integration and run parameter sweeps to investigate how performance scales with P and N on two environments.

II. BACKGROUND

The composite trapezoidal rule approximates $\int_a^b f(x) dx$ with n subintervals of width $h = (b - a)/n$:

$$\int_a^b f(x) dx \approx \frac{h}{2} \left[f(a) + 2 \sum_{i=1}^{n-1} f(a + ih) + f(b) \right]. \quad (1)$$

Parallelization with MPI partitions the interval among P processes. Each process computes a local sum; results are reduced to the root via `MPI_Reduce`. Parameters broadcast via `MPI_Bcast` ensure consistent bounds and n across processes.

III. DESIGN

A. Technology Stack

C (OpenMPI 4.1.1), Python (Matplotlib/NumPy/Pandas), Bash/Slurm on Expanse. The project includes trapezoidal integration, sweep driver, and Slurm job submission.

B. Pseudo Code for MPI Trapezoidal Integration

```
MPI_Init
if rank==0:
    read input a, b, n
MPI_Bcast a, b, n to all ranks
h = (b - a) / n
local_n = n / P (last rank takes remainder)
local_a = a + rank * local_n * h
local_b = local_a + local_n * h
local_sum = trapezoidal(local_a, local_b,
local_n)
```

```
MPI_Reduce(local_sum -> total_sum, root=0, op=SUM)
if rank==0:
    print total_sum and timing
MPI_Finalize
```

C. Sweep Procedure Pseudo Code

```
for N in N_values:
    for P in process_counts:
        run mpirun -np P ./mpi_trap_modified -n N
        parse and record time
compute speedup = T_serial / T_parallel
compute efficiency = speedup / P
save CSV, plot timing/speedup/efficiency
```

IV. SETUP AND IMPLEMENTATION

A. MPI Trapezoidal Integration (C Program)

The core program (`mpi_trap_modified.c`) executes the trapezoidal rule in parallel. Key stages:

- 1) **Initialization:** `MPI_Init`, ranks/size discovery.
- 2) **Parameter Handling:** Rank 0 reads (a, b, n) ; broadcasts to all ranks.
- 3) **Work Division:** Even block partitioning of n ; last rank takes remainder.
- 4) **Local Computation:** Each rank accumulates trapezoid areas on $[local_a, local_b]$.
- 5) **Reduction:** `MPI_Reduce` sums partials into final result at rank 0.
- 6) **Finalize:** `MPI_Finalize`.

B. Parameter Sweep (Python Script)

The sweep script (`mpi_trap_sweep.py`) runs across N and P , times executions, computes speedup/efficiency, writes CSV, and generates plots.

C. Slurm Job Script (Expanse)

The script (`run_trap_debug.sh`) loads modules, compiles with `mpicc`, and launches runs on the debug partition with `mpirun`. The full assignment command is:

```
python mpi_trap_sweep.py 10 20
100000000 2000000000 100000000 1
128 1
```

V. RESULTS

A. Local Results

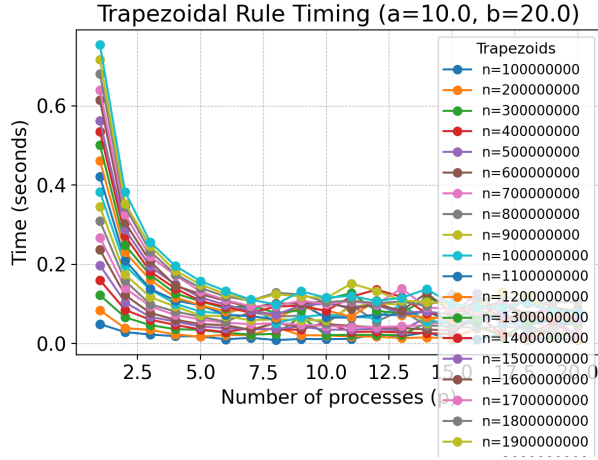


Fig. 1. Local — Execution time vs. processes.

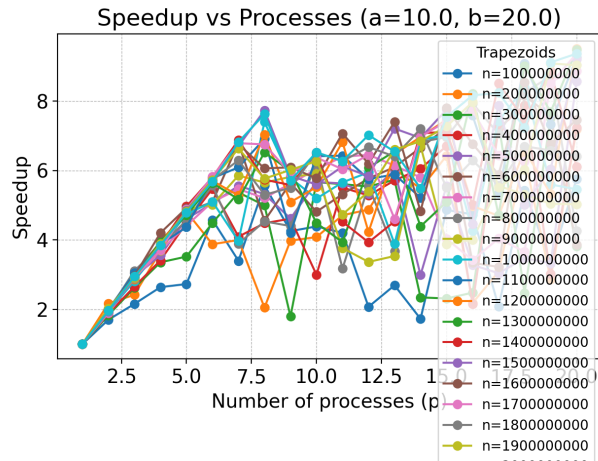


Fig. 2. Local — Speedup vs. processes.

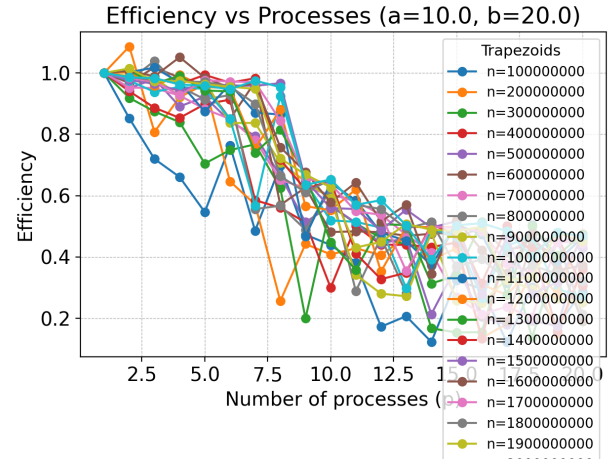


Fig. 3. Local — Efficiency vs. processes.

B. Expanse Results

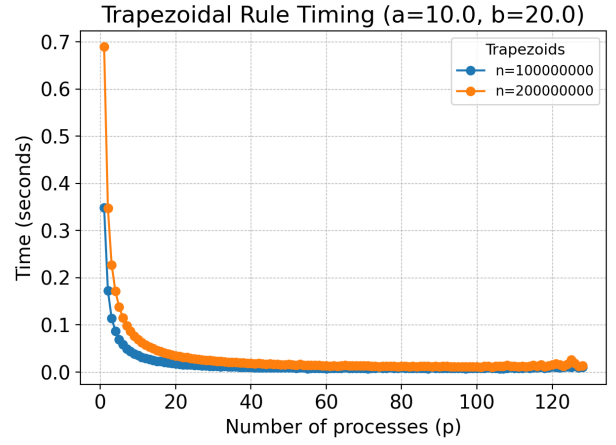


Fig. 4. Expanse — Execution time vs. processes.

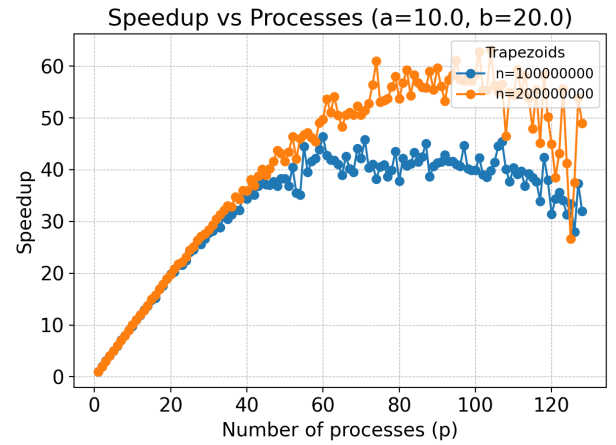


Fig. 5. Expanse — Speedup vs. processes.

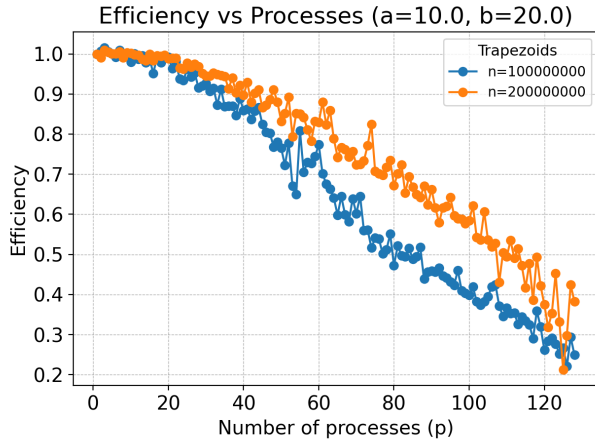


Fig. 6. Expanse — Efficiency vs. processes.

VI. SYSTEM HARDWARE (LOCAL)

CPU	Intel Core i7-14700K, 20C/28T, up to 5.6 GHz
Cache	L1: 768 KiB, L2: 28 MiB, L3: 33 MiB
Memory	62 GiB installed, 49 GiB available
Storage	1 TB NVMe SSD (KINGSTON SFYRS1000G)
OS	Pop!_OS 22.04 LTS, Linux kernel 6.16
GPU	Intel a780 + AMD/ATI 744c

TABLE I

LOCAL SYSTEM HARDWARE SUMMARY.

VII. CONCLUSIONS

Results demonstrate expected strong-scaling behavior: good speedup at moderate P and diminishing returns later. Larger N improves efficiency by increasing compute/communication ratio.

REFERENCES

- [1] SDSC Expanse User Guide, https://www.sdsc.edu/systems/expanse/user_guide.html.
- [1] MPI Forum, “Message Passing Interface Standard,” <https://www.mpi-forum.org/>.
- [2] M. J. Quinn, *Parallel Programming in C with MPI and OpenMP*. McGraw-Hill, 2003.